Terraform Interview question

1. What is Terraform and why do companies use it?

Terraform is an **Infrastructure as Code (IaC)** tool used to define and manage infrastructure using configuration files. Instead of manually creating resources in cloud consoles, engineers write code that describes what infrastructure should exist.

Companies use Terraform because it provides **consistency, automation, version control, and repeatability**. The same code can be used for dev, staging, and production, reducing human errors and making infrastructure changes easier to review and track.

---

2. How is Terraform different from CloudFormation or ARM templates?

Terraform is **cloud-agnostic**, meaning it works with AWS, Azure, GCP, Kubernetes, and many other platforms using a single tool. CloudFormation and ARM templates are limited to AWS and Azure respectively.

Terraform also has a powerful **plan feature**, strong **state management**, and a large provider ecosystem, which makes it suitable for multi-cloud and hybrid environments.

---

3. What is Terraform state and why is it important?

Terraform state is a file that **maps Terraform resources to real infrastructure**. It stores resource IDs, dependencies, and current configuration values.

Terraform uses the state file to decide what needs to be created, updated, or destroyed. Without state, Terraform would not know whether infrastructure already exists, making safe changes impossible.

---

4. What problems occur if Terraform state is not managed properly?

If state is not handled correctly, Terraform can create duplicate resources, delete existing infrastructure, or overwrite changes made by others. State conflicts often happen when multiple engineers work on the same setup without proper locking.

That's why production systems always use **remote state with locking**.

---

5. What is a Terraform backend?

A backend defines **where and how Terraform stores the state file**. The default backend stores state locally, but real projects use remote backends like object storage services.

Remote backends allow **team collaboration, state locking, security, and backups**, making them essential for production use.

---

6. What is state locking and why is it needed?

State locking ensures that **only one Terraform operation can modify the state at a time**. This prevents multiple applies from running simultaneously.

Without locking, parallel changes can corrupt state or cause unpredictable infrastructure behavior.

---

7. What is the Terraform workflow?

Terraform follows a clear workflow:

1. terraform init – prepares the project
2. terraform plan – shows what will change
3. terraform apply – applies the changes

This workflow allows engineers to **review changes before execution**, reducing production risk.

---

8. What happens internally during terraform init?

During initialization, Terraform downloads required providers, configures the backend, and prepares state storage. It also creates the .terraform directory.

This step ensures Terraform has everything it needs before planning or applying changes.

---

9. What is the purpose of terraform plan?

terraform plan previews infrastructure changes by comparing the desired configuration with the current state and real infrastructure.

It helps engineers catch unexpected changes before applying them, making it a critical safety step.

---

10. What is idempotency in Terraform?

Idempotency means running the same Terraform code multiple times results in the **same infrastructure state**.

Terraform achieves this by checking current state and applying only required changes, which is essential for automation and repeated executions.

---

11. What are Terraform providers?

Providers are plugins that allow Terraform to communicate with external APIs such as cloud platforms or services.

They translate Terraform configuration into API calls, enabling Terraform to manage real infrastructure resources.

---

12. What is a Terraform resource?

A resource is a block that represents **one infrastructure component** like a VM, network, or storage bucket.

Resources are declarative, meaning you define the desired state and Terraform handles the creation and updates.

---

13. What is a Terraform module and why is it important?

A module is a reusable collection of Terraform resources grouped together for a specific purpose.

Modules reduce duplication, enforce standards, and make large Terraform codebases easier to maintain and scale.

---

14. Difference between root module and child module?

The root module is the main directory where Terraform commands are run. Child modules are reusable units called from the root module.

In production setups, the root module remains small while most logic lives in child modules.

---

15. What are input variables in Terraform?

Input variables allow Terraform code to be **parameterized**, making it flexible and reusable.

They help separate configuration values from logic and support multiple environments without changing the code.

---

16. What are output values used for?

Outputs expose important information such as IP addresses or resource identifiers.

They are useful for debugging, integrations, and passing values between Terraform modules.

---

17. What is terraform.tfvars file?

The terraform.tfvars file provides values for input variables.

It keeps environment-specific values separate from the main configuration, improving clarity and reusability.

---

18. What are locals in Terraform?

Locals are named expressions used to store calculated or repeated values.

They improve readability and reduce duplication, especially for naming conventions or complex expressions.

---

19. What is dependency management in Terraform?

Terraform automatically determines resource dependencies based on references between resources.

This allows Terraform to create, update, and delete resources in the correct order without manual intervention.

---

20. What is depends_on and when should it be used?

depends_on explicitly tells Terraform that one resource depends on another.

It should be used only when Terraform cannot automatically infer the dependency, such as with indirect or external relationships.

21. What are data sources in Terraform?

Data sources allow Terraform to **read information about existing infrastructure** instead of creating it.

They are used when:

- A resource already exists
- Terraform needs its details (IDs, CIDR, AMI, etc.)

Data sources **do not create or modify** infrastructure.

---

22. Difference between resource and data source?

- **Resource** → Creates or manages infrastructure
- **Data source** → Reads existing infrastructure only

A resource changes the real world, while a data source is **read-only**.

---

23. What is terraform refresh?

terraform refresh updates the Terraform state file with the **current real infrastructure values**.

Earlier it was used to detect drift, but now Terraform does this automatically during plan and apply, so refresh is rarely used directly.

---

24. What is infrastructure drift?

Drift happens when infrastructure is **changed manually outside Terraform**, such as using a cloud console.

Terraform detects drift during terraform plan and tries to bring infrastructure back to the state defined in code. Drift is risky because it breaks consistency and predictability.

---

25. How do you handle multiple environments in Terraform?

Common approaches include:

- Separate directories per environment
- Separate state files
- Separate variable files

The safest approach is **separate state per environment**, so mistakes in one environment don't affect others.

---

26. What are Terraform workspaces?

Workspaces allow multiple state files for the **same Terraform configuration**.

They are useful for small setups, but in large or production systems, they can cause confusion because environment separation is not very visible.

---

27. What is count in Terraform?

count is used to create **multiple copies of a resource**.

It works well for simple cases, but it relies on numeric indexes, which can cause resource recreation issues when values change.

---

28. What is for_each and why is it better than count?

for_each creates resources using **maps or sets**, giving each resource a unique key.

It is better than count because:

- Resource identity is stable
- Less risk of accidental deletion
- Easier to manage changes

---

29. What is a lifecycle block?

The lifecycle block controls **how Terraform manages a resource**, not what it creates.

It is used for:

- Preventing deletion
- Ignoring changes
- Controlling creation order

Commonly used to protect critical resources.

---

30. What does create_before_destroy do?

This lifecycle setting tells Terraform to **create a new resource before deleting the old one**.

It helps reduce or avoid downtime, especially for resources like load balancers or instances behind traffic routing.

---

31. Difference between terraform apply and terraform apply -auto-approve?

- terraform apply → shows the plan and waits for confirmation
- terraform apply -auto-approve → applies changes immediately

Auto-approve removes the final safety check and is usually used only in automated pipelines, not manual production changes.

---

32. What is terraform destroy and when should it be used carefully?

terraform destroy removes **all resources managed by Terraform**.

It is useful for test or temporary environments but very dangerous in production. One wrong destroy can wipe out critical infrastructure.

---

33. How does Terraform know what to delete or modify?

Terraform compares:

1. Terraform configuration
2. Terraform state
3. Real infrastructure

Based on differences, it decides what to create, update, or destroy. Accurate state is essential for correct decisions.

---

34. What is drift detection and how does Terraform handle it?

Terraform detects drift during terraform plan by comparing real infrastructure with state.

When drift is found, Terraform shows it in the plan and tries to align infrastructure back to the desired state defined in code.

---

35. Why should production infrastructure not be changed manually?

Manual changes bypass:

- Version control
- Code reviews
- Rollbacks

They also cause drift and confusion. In production, infrastructure should always be changed through Terraform code.

---

36. What is the purpose of .terraform.lock.hcl file?

This file locks **exact provider versions** used in the project.

It ensures all engineers and pipelines use the same provider versions, avoiding unexpected behavior caused by provider updates.

---

37. Why is provider version pinning important?

Providers change over time and may introduce breaking changes.

Pinning provider versions ensures:

- Predictable behavior
- Controlled upgrades
- Stability in production systems

---

38. What is a Terraform workspace and what problem does it solve?

Workspaces allow multiple environments using the same code by maintaining separate state files.

They solve simple environment separation but are less suitable for complex or large production setups due to visibility and safety concerns.

---

39. Why are Terraform workspaces not recommended for large teams?

Workspaces hide environment differences inside Terraform instead of making them explicit.

This increases the risk of:

- Applying changes to the wrong environment
- Confusion during reviews

Explicit separation is safer for large teams.

---

40. What is terraform import and when is it used?

terraform import brings **existing infrastructure into Terraform state**.

It does not create resources; it only maps them into state. After import, Terraform code must be written to match the imported resource.

41. Why is terraform import considered risky if not handled properly?

terraform import only updates the **state file**, not the Terraform configuration.
If the code does not correctly match the imported resource, Terraform may later try to modify or even destroy it. That's why after importing, engineers must carefully write and verify the Terraform code before running apply.

---

42. What is a null resource and why is it controversial?

A null resource does not manage real infrastructure. It is often used to trigger scripts or commands.
It is controversial because it mixes **imperative behavior** into Terraform's declarative model, making code harder to maintain and debug.

---

43. What are provisioners and why are they discouraged?

Provisioners run scripts during resource creation or destruction.
They are discouraged because they:

- Are not idempotent
- Have weak error handling
- Make Terraform behavior unpredictable

Terraform is meant for provisioning infrastructure, not configuring software inside it.

---

44. When is using a provisioner acceptable?

Provisioners are acceptable **only as a last resort**, such as bootstrapping legacy systems or performing tasks that cannot be handled by other tools. Even then, they should be minimal and well-documented.

---

45. Difference between local-exec and remote-exec?

- local-exec runs commands on the machine where Terraform is executed.
- remote-exec runs commands on the target resource itself.

Both should be used cautiously because they introduce imperative logic.

---

46. How does Terraform handle secrets and sensitive data?

Terraform allows marking variables as **sensitive**, which hides them from CLI output.
However, sensitive values still exist in the **state file**, so state must be encrypted and access-controlled. In production, secrets are usually pulled from secret managers.

---

47. Why should Terraform state be encrypted?

Terraform state can contain:

- Passwords
- Keys
- Internal infrastructure details

Encrypting the state protects sensitive information from unauthorized access, especially when using remote backends shared across teams.

---

48. How do you manage Terraform state in a team environment?

Teams typically:

- Use a remote backend
- Enable state locking
- Restrict access using roles
- Apply changes through controlled processes

This avoids conflicts and accidental infrastructure damage.

---

49. What is terraform taint and why is it rarely used now?

terraform taint marks a resource for recreation on the next apply.
It is rarely used today because Terraform usually detects unhealthy resources automatically, and manual tainting can be risky if misused.

---

50. What is terraform state rm and when is it used?

This command removes a resource from Terraform state **without deleting the actual resource**.

It is used when Terraform should stop managing a resource, but must be used carefully since Terraform will no longer track that resource.

---

51. What is terraform state mv?

terraform state mv moves resources within the state file.
It is commonly used during refactoring, such as renaming resources or moving them between modules, without causing resource recreation.

---

52. Why is refactoring Terraform code risky without state commands?

If resources are renamed or moved in code without updating state, Terraform may think they were deleted and recreate them.
State commands ensure Terraform understands that resources were **moved, not destroyed**.

---

53. Difference between terraform plan and terraform plan -out?

- terraform plan shows changes on screen
- terraform plan -out saves the plan to a file

Saved plans ensure that exactly the reviewed changes are later applied.

---

54. Why is applying a saved plan important in CI/CD?

Applying a saved plan guarantees that **what was reviewed is exactly what gets applied**.
This improves trust, auditability, and safety in automated workflows.

---

55. How does Terraform behave when a resource creation partially fails?

Terraform may create some resources and fail on others.
It records partial state and stops. Engineers must review the state and fix the issue before reapplying, rather than blindly retrying.

---

56. What is a partial apply and why is it dangerous?

A partial apply leaves infrastructure in an **inconsistent state**.
This can cause dependencies to break or future applies to behave unexpectedly. Careful plan review and error handling are critical.

---

57. What is the role of CI/CD in Terraform workflows?

CI/CD enforces:

- Formatting
- Validation
- Security checks
- Approvals

It ensures Terraform changes follow the same discipline as application code, reducing human error.

---

58. What validations should be done before running terraform apply?

Common validations include:

- terraform fmt
- terraform validate
- Policy checks
- Security scanning

These steps catch errors early before affecting real infrastructure.

---

59. What is terraform validate?

terraform validate checks if the configuration is syntactically correct and internally consistent.
It does not contact the cloud but helps catch mistakes early.

---

60. Why is formatting important in Terraform code?

Consistent formatting improves readability, reduces review noise, and enforces standards across teams.
terraform fmt ensures everyone follows the same style automatically.

61. What is policy as code in Terraform and why is it important?

Policy as code means **automatically enforcing rules** on infrastructure instead of relying on manual reviews.
It ensures things like encryption, tagging, and allowed resource types are checked before resources are created. This is important because manual checks don't scale as teams grow.

---

62. Why is a tagging strategy important in Terraform projects?

Tags help with **cost tracking, ownership, security, and automation**.
Without consistent tags, teams can't easily identify who owns a resource or why it exists.
Terraform makes it easy to enforce tagging consistently across all resources.

---

63. How do you enforce mandatory tags using Terraform?

Mandatory tags are enforced by defining a **common set of tags** using variables or locals and merging them into every resource.
In mature setups, policy checks ensure Terraform fails if required tags are missing.

---

64. Difference between hardcoding values and using variables?

Hardcoded values make Terraform code rigid and hard to reuse.
Using variables makes the same code work across multiple environments by changing inputs, improving flexibility and maintainability.

---

65. What is variable validation and why is it useful?

Variable validation allows you to define **rules for acceptable input values**.
It prevents invalid or unsafe values from being used, catching mistakes early before infrastructure is affected.

---

66. Difference between default variables and required variables?

- **Default variables** have fallback values and are optional.
- **Required variables** must be explicitly provided.

Defaults are used for safe values, while critical inputs are usually required.

---

67. How do you structure Terraform code for large projects?

Large projects use:

- Reusable modules
- Separate environments
- Clear directory structure

The root module stays small and calls modules for networking, compute, and security, improving long-term maintainability.

---

68. Why is keeping the Terraform root module small a best practice?

A small root module improves clarity and reduces risk.
It acts as a coordinator while reusable logic lives in modules, making changes safer and easier to understand.

---

69. How do you handle environment-specific values cleanly?

Environment-specific values are handled through:

- Separate variable files
- Separate environment directories

This avoids complex conditional logic and keeps configurations clear and predictable.

---

70. Why is conditional logic in Terraform considered risky?

Too much conditional logic makes Terraform code harder to read and reason about.
It can also cause unexpected resource creation or deletion. Simpler, explicit configurations are safer.

---

71. What is the Terraform console and when is it useful?

Terraform console is an interactive shell used to **evaluate expressions and variables**.
It's useful for debugging complex logic and testing expressions without running a full plan.

---

72. What are Terraform functions and why are they important?

Terraform functions help manipulate strings, lists, maps, and numbers.
They reduce duplication and help build dynamic but readable configurations.

---

73. Why should Terraform code be readable rather than clever?

Readable code is easier to maintain, review, and debug.
Clever but complex logic increases risk during incidents when clarity matters most.

---

74. What is terraform fmt and why should it be enforced?

terraform fmt automatically formats Terraform code using standard rules.
Enforcing it ensures consistency and reduces unnecessary review discussions.

---

75. How does Terraform handle parallel resource creation?

Terraform creates independent resources **in parallel** to speed up execution.
Dependencies determine the order. Misconfigured dependencies can cause failures.

---

76. Why do some Terraform applies take a long time?

Long applies usually happen due to:

- Large numbers of resources
- Slow cloud APIs
- Sequential dependencies

Understanding dependencies helps optimize performance.

---

77. What is the impact of API rate limits on Terraform?

Cloud providers limit API requests.
When Terraform exceeds these limits, operations slow down or fail. Large deployments must account for this.

---

78. What is the -parallelism flag and when should it be adjusted?

The -parallelism flag controls how many resources Terraform creates at once.
Lowering it helps avoid API throttling or instability in large or sensitive environments.

---

79. Why is Terraform sometimes blamed for cloud failures incorrectly?

Terraform executes API calls, but failures often come from cloud provider limits, misconfigurations, or dependencies.
Terraform surfaces the issue—it doesn't create the problem.

---

80. How do you safely test Terraform changes before production?

Changes are tested using:

- Separate environments
- Plan reviews
- Automated checks
- Sometimes sandbox accounts

Applying changes directly to production without testing is unsafe.

81. What is the role of version control in Terraform projects?

Version control keeps a **history of infrastructure changes**, allows collaboration, and enables rollback.
Every Terraform change should be tracked just like application code to ensure accountability and auditability.

---

82. Why should Terraform code go through code reviews?

Code reviews catch mistakes, enforce standards, and share knowledge across the team.
Because infrastructure changes can have large impact, peer review is critical for safety.

---

83. What is infrastructure immutability and how does Terraform support it?

Immutability means replacing resources instead of modifying them in place.
Terraform supports this through lifecycle rules and recreation logic, reducing drift and unpredictable behavior.

---

84. Why do some Terraform changes force resource recreation?

Some cloud resources don't support in-place updates.
Terraform clearly shows these replacements in the plan so engineers can understand the impact before applying.

---

85. How do you prevent accidental destruction of critical resources?

Critical resources are protected using:

- prevent_destroy lifecycle rule
- Strict access controls
- Separate state files

These safeguards reduce the risk of catastrophic mistakes.

---

86. What is the role of backups in Terraform-managed infrastructure?

Terraform manages infrastructure configuration, not data.
Backups are still required for databases, storage, and state files to recover from data loss.

---

87. Why is separating state per environment important?

Separate state files isolate environments.
This ensures mistakes in development or testing never affect production infrastructure.

---

88. How do you handle breaking changes in Terraform providers?

Provider upgrades are tested in non-production environments first.
Versions are upgraded deliberately and reviewed carefully before being used in production.

---

89. Why should Terraform upgrades be planned and tested?

Terraform core and provider upgrades can change behavior.
Planning and testing upgrades prevents unexpected failures in existing pipelines and infrastructure.

---

90. What mindset should a 3–6 year engineer have while using Terraform?

Engineers should focus on **safety, predictability, and long-term maintainability**, not just making Terraform work.
Good Terraform usage is about reducing risk, not writing clever code.

---

91. What is infrastructure modularity and why does it matter?

Modularity means breaking infrastructure into small, reusable modules.
It improves readability, reduces duplication, and allows teams to change one part without impacting others.

---

92. Why do poorly designed modules cause long-term problems?

Poor modules have unclear responsibilities, too many inputs, or environment-specific logic.
Over time, they become hard to reuse and maintain, leading to duplication and technical debt.

---

93. What makes a Terraform module "good" in real projects?

A good module has:

- One clear responsibility
- Minimal, meaningful inputs
- Clear outputs
- No environment-specific logic

Good modules are predictable and boring—in a good way.

---

94. Why is backward compatibility important when changing modules?

Modules are often shared across teams and environments.
Breaking changes can affect many systems at once, so backward compatibility allows safer, gradual upgrades.

---

95. What is semantic versioning and how does it apply to Terraform modules?

Semantic versioning communicates risk:

- Patch → bug fixes

- Minor → backward-compatible features
- Major → breaking changes

It helps teams upgrade modules with confidence.

---

96. Why should Terraform modules be versioned explicitly?

Explicit versioning prevents unexpected changes.
Without version pinning, a module update could change infrastructure behavior without warning.

---

97. How do you safely upgrade Terraform modules in production?

Upgrades are tested in lower environments first.
Engineers review the plan carefully and roll out changes gradually to avoid outages.

---

98. Difference between immutable and mutable infrastructure?

Immutable infrastructure replaces resources for changes.
Mutable infrastructure modifies existing resources in place.
Immutable approaches are generally safer and more predictable.

---

99. Why does Terraform sometimes recreate resources instead of updating them?

Some resources can't be updated in place due to platform limitations.
Terraform shows this clearly in the plan so engineers can prepare for the impact.

---

100. How do you reduce downtime caused by resource recreation?

Downtime is reduced by:

- create_before_destroy
- Load balancers
- Rolling deployment strategies

Infrastructure should always be designed assuming recreation can happen.